

FedCoLLM: A Parameter-Efficient Federated Co-tuning Framework for Large and Small Language Models

Tao Fan^{1,2*}, Yan Kang², Guoqiang Ma², Lixin Fan²,
Kai Chen¹, and Qiang Yang^{1,2}

¹ Department of Computer Science and Engineering, HKUST, Hong Kong
{tfanac, kaichen, qyang}@cse.ust.hk

² WeBank, China
{yangkang, zotrseeewma, lixinfan}@webank.com

Abstract. By adapting Large Language Models (LLMs) to domain-specific tasks or enriching them with domain-specific knowledge, we can fully harness the capabilities of LLMs. Nonetheless, a gap persists in achieving simultaneous mutual enhancement between the server’s LLM and the downstream clients’ Small Language Models (SLMs). To address this, we propose FedCoLLM, a novel and parameter-efficient federated framework designed for co-tuning LLMs and SLMs. This approach is aimed at adaptively transferring server-side LLMs knowledge to clients’ SLMs while simultaneously enriching the LLMs with domain insights from the clients. To accomplish this, FedCoLLM utilizes lightweight adapters in conjunction with SLMs, facilitating knowledge exchange between server and clients in a manner that respects data privacy while also minimizing computational and communication overhead. **Our evaluation of FedCoLLM, utilizing various public LLMs and SLMs across a range of NLP text generation tasks,** reveals that the performance of clients’ SLMs experiences notable improvements with the assistance of the LLMs. Simultaneously, the LLMs enhanced via FedCoLLM achieves comparable performance to that obtained through direct fine-tuning on clients’ data.

Keywords: LLMs · Federated Learning · Parameter-Efficient

1 Introduction

The emergence of Large Language Models (LLMs) has profoundly transformed the landscape of artificial intelligence. In particular, cutting-edge LLMs like GPT-4 [13] have garnered significant attention due to their exceptional performance across a range of natural language generation tasks. This development has spurred the release of numerous high-performance open-source LLMs, such as LLaMa [18], OPT [21], greatly promoting the commercial application of LLMs technology. Despite their widespread success in various general NLP tasks, LLMs

* Corresponding author

face limitations that hinder their adoption in domain-specific applications. The primary challenges include:

- **Domain-Specific Knowledge Privacy.** When downstream clients are unable to access the LLMs parameter, they have to send their labeled data to the LLMs owners for fine-tuning. This process inevitably discloses the privacy of clients’ sensitive domain-specific data.
- **Constrained Resources.** Even when downstream enterprises can obtain the model parameters of LLMs, they often encounter significant resource constraints. Fine-tuning these LLMs requires substantial computing and storage resources, posing a barrier to adoption by small and medium-sized companies with limited resources. As a result, these companies are restricted to fine-tuning Small Language Models(SLMs) using their domain-specific data.
- **Mutual Knowledge Transfer Between LLMs and SLMs.** Optimizing both LLMs and SLMs in different parties establishes a positive feedback loop that allows both LLMs and SLMs to evolve continuously. Initially, the LLMs on the server can disseminate general knowledge and capabilities to the clients’ SLMs, which are then trained on domain-specific data for downstream applications. Subsequently, these domain-specific SLMs can contribute their industry-specific knowledge back to the server’s LLMs. This transfer of specialized knowledge serves to enhance the LLMs’ understanding and capabilities, broadening its scope and depth. Nevertheless, mutually enhancing the server’s LLMs and clients’ SLMs is rarely exploited in literature.

To address the aforementioned challenges, we propose FedCoLLM, an innovative and parameter-efficient federated co-tuning framework for LLMs and SLMs. This framework is designed to enhance the performance of both server-side LLMs and client-side SLMs. As illustrated in Figure 1, FedCoLLM deploys a LLM on the server and introduces a SLM to serve as a bridge between the privacy data in the clients and LLM in the server. The SLM operates simultaneously across multiple clients and the server, facilitating efficient communication and collaboration. FedCoLLM offers three distinct advantages:

- **Efficient Computation and Communication.** FedCoLLM initially runs the SLM under standard federated learning(FL) frameworks [10,20]. This approach integrates a parameter-efficient adapter module, such as LoRA [9], significantly reducing the computation and communication costs associated with FedCoLLM.
- **Enhanced Data Privacy.** By leveraging the FL framework to fine-tune the SLM, FedCoLLM fully utilizes the FL security protection mechanisms(such as SecureAggregation [2]) to preserve the privacy of clients’ data. This ensures that sensitive information remains protected during the fine-tuning process.
- **Knowledge Transfer and Mutual Enhancement.** FedCoLLM employs knowledge distillation(KD) techniques [8], to transfer knowledge between the LLM and SLM on the server. This process is facilitated by an auxiliary distillation dataset, making it particularly beneficial for clients with limited

resources. Through this knowledge transfer, both the server LLM and client SLMs are mutually enhanced, leading to improved overall performance.

Extensive experiments conducted on various LLMs and SLMs, including GPT-2 [15], OPT [21], and LLaMa2 [18], demonstrate the competitive performance of our FedCoLLM framework across a range of NLP text generation tasks. The results show that the SLMs can achieve significant enhancements with the support of the LLM, while the LLM can deliver comparable results to fine-tuning with all clients’ domain data directly. Importantly, our framework is more resource-efficient, requiring lower computation and communication costs.

2 Related Work

2.1 Knowledge Distillation

Knowledge distillation is a technique that has gained significant attention in recent years, as it enables the transfer of knowledge from a larger teacher model to a smaller student model. One of the early works in this area was proposed by [8], which introduced the concept of knowledge distillation and demonstrated its effectiveness in improving the performance of compressed models. Since then, numerous studies have built upon this foundation and explored various distillation strategies [7]. For instance, [4,11] improved response-based knowledge distillation, which let the student model directly mimic the final prediction of the teacher model. [1] proposed FitNets, which focus on matching intermediate representations between the teacher and student models. Another notable work is the relational knowledge distillation approach introduced by [14], which captures pairwise relationships between outputs to enhance distillation efficiency. Different from one-way knowledge distillation between the teacher and student networks, Deep Mutual Learning (DML) [22] allows two networks to learn from each other through their predicted probability distributions during the training process. These studies have demonstrated the potential of knowledge distillation in various tasks, such as image classification, object detection, and natural language processing.

2.2 Federated Learning for Large Language Models

Parameter-Efficient Fine-Tuning (PEFT) techniques [9] provide a straightforward remedy to address the challenges of communication overhead and fine-tuning expenses in Federated Learning (FL) for Large Language Models (LLMs) [16]. Numerous investigations have extended the application of PEFT methods within the FL framework tailored for LLMs. Notable contributions include FedPETuning [23], Federated Adapter Tuning [3], and Federated Prompt Tuning [24]. These research outcomes suggest that FL clients, particularly those with constrained storage capacities like mobile devices, can significantly profit from the adoption of PEFT methods. These approaches facilitate the sharing of LLMs across various tasks while necessitating the retention of only a minimal set of parameters per

task, effectively decreasing storage demands. Through the utilization of PEFT methods, FL clients can adeptly tailor LLMs to meet their unique requirements, all the while minimizing communication overhead and reducing fine-tuning costs.

3 The Proposed Method

In this section, we present a comprehensive overview of our federated co-tuning LLMs and SLMs framework, termed FedCoLLM. This framework is based on parameter-efficient fine-tuning (PEFT) and knowledge distillation techniques. We begin by defining the specific problem addressed in this study, followed by a detailed introduction to our approach. Finally, we delve into the computational and communication complexities, as well as the privacy-preserving analysis, of our FedCoLLM framework.

3.1 Problem Definition

In this work, we consider the federated learning setting in which the **server owns an LLM f_ψ parameterized by ψ and K clients that each client k has a SLM g_ϕ parameterized by ϕ** . The server and clients aim to collaboratively enhance the performance of the LLM and SLMs without sharing private data through federated learning. Specifically,

- Each client possesses its own local private dataset $\mathcal{D}^k$. Clients aim to collectively train a global SLM g_ϕ based on their local models initialized with an SLM (e.g., LLaMa2-1.3B [19]) without divulging their private data. The objective can be formulated as follows:

$$\min_{\phi} \mathcal{L}_1(\phi; \{\mathcal{D}^k\}_{k=1}^K) \quad (1)$$

- The server owns an auxiliary dataset D^a . The server aims to transfer knowledge between its owned LLM f_ψ and the global SLM g_ϕ aggregated from clients' local SLMs to enhance both the LLM and SLMs. The objective can be formulated as follows:

$$\min_{\phi, \psi} \mathcal{L}_2(\phi, \psi; D^a) \quad (2)$$

We regard the server as semi-honest. FedCoLLM solves the optimization problems formulated in Eq.(1) and Eq.(2) in an efficient and privacy-preserving manner. We will elaborate on FedCoLLM in Section 3.2.

3.2 FedCoLLM

FedCoLLM is a novel framework designed to facilitate the collaborative evolution of both server-side LLM and client-side SLMs. The goal of the FedCoLLM is threefold:

- **Collaborative Knowledge Transfer and Adaptation.** The server and clients work together to transfer and adapt the knowledge of the LLM owned by the server. This helps clients build local SLMs that benefit from the server’s LLM knowledge. By leveraging the server’s LLM, clients can improve their local SLMs’ performance without requiring extensive local training data or computational resources.
- **Data Augmentation for the Server’s LLM.** Federated learning also aims to leverage clients’ data to augment and enhance the server’s LLM. Clients’ data often contains valuable local information and patterns that can be used to improve the server model’s generalization and performance. By incorporating this data, the server’s LLM can become more robust and adaptive to different scenarios and domains.
- **Privacy-Preserving and Efficient Knowledge Transfer.** A crucial aspect of federated learning is ensuring that knowledge transfer occurs in a privacy-preserving and efficient manner. Clients’ raw private data should not be directly uploaded to the LLM server, preserving their privacy. Instead, only model updates or aggregated information are shared with the LLM server. Additionally, the knowledge transfer process should be efficient, minimizing communication costs and computational overhead.

Toward this goal, we (1) adopt lightweight LoRA modules as the bridge to transfer the knowledge between clients and the server, (2) leverage mutual knowledge distillation to transfer knowledge between the LLM and the aggregated SLM, and (3) employ secure aggregation to protect the privacy of the knowledge transfer process.

Specifically, we assume that clients and the server share a SLM g_ϕ parameterized by ϕ . Each client k inserts a small low-rank adapter parameterized by θ_k into its local SLM. We denote a client’s local SLM with the added θ as $g_{\phi+\theta}$. Instead of training a global SLM ϕ , clients collaboratively train a global LoRA module θ . Then, Eq.(1) can be reformulated as follows:

$$\mathcal{L}_1(\theta; \{\mathcal{D}^k\}_{k=1}^K) = \frac{1}{K} \sum_{k=1}^K \mathbb{E}_{(x,y) \sim \mathcal{D}^k} \ell_{\text{TA}}^k(g_{\phi+\theta}(x), y). \quad (3)$$

where ℓ_{TA} is the task loss for training the global LoRA module θ . The original model parameter ϕ of each client’s local SLM is frozen during training.

The server inserts a small low-rank adapter parameterized by ω into its LLM f_ψ . We denote the server’s LLM f_ψ with the added ω as $f_{\psi+\omega}$. The server conducts the mutual knowledge transfer between the LLM $f_{\psi+\omega}$ and the global SLM $g_{\phi+\theta}$ through supervised fine-tuning and mutual knowledge distillation based on the auxiliary dataset $\mathcal{D}^a$.

We formulate the losses of supervised fine-tuning $f_{\psi+\omega}$ and $g_{\phi+\theta}$ (denoted as $\mathcal{L}_{\text{FT}}^f$ and $\mathcal{L}_{\text{FT}}^g$) as follows:

$$\begin{aligned} \mathcal{L}_{\text{FT}}^f(\omega; \mathcal{D}^a) &= \mathbb{E}_{(x,y) \sim \mathcal{D}^a} \ell_{\text{CE}}(f_{\psi+\omega}(x), y), \\ \mathcal{L}_{\text{FT}}^g(\theta; \mathcal{D}^a) &= \mathbb{E}_{(x,y) \sim \mathcal{D}^a} \ell_{\text{CE}}(g_{\phi+\theta}(x), y). \end{aligned} \quad (4)$$

where ℓ_{CE} is the cross-entropy loss; the model parameters ψ and ϕ are frozen during fine-tuning.

The mutual knowledge distillation losses for fine-tuning $f_{\psi+\omega}$ and $g_{\phi+\theta}$ models (denoted as $\mathcal{L}_{\text{KD}}^f$ and $\mathcal{L}_{\text{KD}}^g$) are formulated as follows:

$$\begin{aligned}\mathcal{L}_{\text{KD}}^f(\omega; \mathcal{D}^a) &= \mathbb{E}_{x \sim \mathcal{D}^a} \ell_{\text{KL}}(f_{\psi+\omega}(x), g_{\phi+\theta}(x)), \\ \mathcal{L}_{\text{KD}}^g(\theta; \mathcal{D}^a) &= \mathbb{E}_{x \sim \mathcal{D}^a} \ell_{\text{KL}}(g_{\phi+\theta}(x), f_{\psi+\omega}(x)).\end{aligned}\tag{5}$$

where ℓ_{KL} is the Kullback Leibler (KL) divergence function; the model parameters ψ and ϕ are frozen during knowledge distillation.

Combining Eq.(4) and Eq.(5), we formulate the mutual knowledge transfer conducted on the server as follows:

$$\begin{aligned}\mathcal{L}_2(\theta, \omega; \mathcal{D}^a) &= \mathcal{L}^f(\omega; \mathcal{D}^a) + \mathcal{L}^g(\theta; \mathcal{D}^a), \\ \text{in which} \\ \mathcal{L}^f(\omega; \mathcal{D}^a) &= \mathcal{L}_{\text{FT}}^f(\omega; \mathcal{D}^a) + \lambda \mathcal{L}_{\text{KD}}^f(\omega; \mathcal{D}^a), \\ \mathcal{L}^g(\theta; \mathcal{D}^a) &= \mathcal{L}_{\text{FT}}^g(\theta; \mathcal{D}^a) + \lambda \mathcal{L}_{\text{KD}}^g(\theta; \mathcal{D}^a).\end{aligned}\tag{6}$$

where λ is the hyperparameter that controls the weight of mutual knowledge transfer.

After mutual knowledge transfer, the global LoRA module θ is distributed to all clients, which in turn adopts Eq.(1) to further train θ based on their local datasets.

FedCoLLM thus fosters a symbiotic relationship between the server and clients, where both parties benefit from the collective knowledge and expertise encoded in their respective language models. By leveraging the complementary strengths of server-side LLMs and client-side SLMs, FedCoLLM paves the way for more efficient and effective federated learning in the realm of natural language processing, enabling the collaborative evolution of LLM and SLMs. **We illustrate the FedCoLLM in Figure 1 and describe the associated training algorithm in Algorithm 1. The workflow of FedCoLLM proceeds as follows:**

1. In the t -th communication round, the server broadcasts the SLM $g_{\phi+\theta}$ global adapter θ to K clients. Each client k then replaces its local adapter θ_k with the received global adapter θ .
2. During local training, the K clients fine-tune their respective local adapters using their private data. This step allows the clients to adapt their models to their specific data distributions while preserving the knowledge encoded in the global adapter.
3. After local training, the K clients send their respective local adapters to the server. The server SLM $g_{\phi+\theta}$ then aggregates these local adapters using a secure averaging technique, such as SecureAvg, and updates the global adapter θ in the SLM accordingly.
4. On the server side, LLM $f_{\psi+\omega}$ and SLM $g_{\phi+\theta}$ engage in knowledge distillation. This process involves transferring knowledge between the two models with the aid of an auxiliary distillation dataset. Through this distillation, both models

can benefit from each other’s learned representations, leading to improved performance and adaptability.

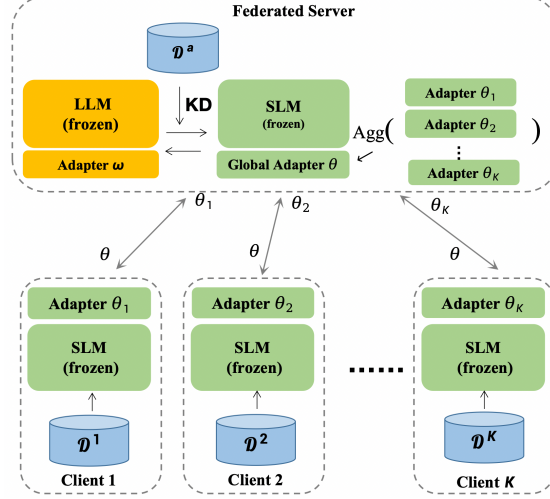


Fig. 1: FedCoLLM (Federated parameter-efficient co-tuning of clients’ domain SLMs and the server’s LLMs. Clients’ SLMs learn from each other via federated fine-tuning of their adapter modules and transfer knowledge from and to the server’s LLM)

3.3 Computation and Communication Complexity

One key advantage of FedCoLLM is its computational efficiency. By utilizing PEFT, it markedly decreases the parameters needing fine-tuning updates. Furthermore, the server-side distillation process compresses knowledge from local models into a smaller global model, optimizing computational resources. This enables effective learning from all clients’ collective data while keeping the model size manageable. In terms of communication complexity, FedCoLLM minimizes the amount of data exchanged between clients and the server. Instead of transmitting entire models or large datasets, clients only share their locally fine-tuned model updates with the server. This approach significantly reduces communication overhead.

3.4 Privacy-Preserving Analysis

FedCoLLM is meticulously designed with privacy preservation as its foundation. Recognizing the importance of data confidentiality, the framework ensures clients

never directly disclose raw local data. Privacy protection is further enhanced through PEFT and knowledge distillation, minimizing sensitive information exposure during training. By using knowledge distillation, FedCoLLM transfers key insights to a unified global model, sharing only aggregated, non-sensitive knowledge and preserving individual client privacy. Additionally, it seamlessly integrates with the standard FL framework for SLMs fine-tuning, leveraging security mechanisms like SecureAggregation [2] to maintain robust data privacy for all clients.

Algorithm 1 FedCoLLM

Input:
1: K : number of clients;
2: T : total number of rounds;
3: $\mathcal{D}^a$: the auxiliary distillation dataset;
4: $\mathcal{D}^k$: local datasets for each client k ;
5: η_ω : the learning rate for optimizing LLM $f_{\psi+\omega}$;
6: η_θ : the learning rate for optimizing SLM $g_{\phi+\theta}$.
Output: $f_{\psi+\omega}$, $g_{\phi+\theta}$.
7: **for** $t = 1, 2, \dots, T$ **do**
8: // **Server side:**
9: Broadcast the global adapter θ^t to all K clients.
10: $\theta_k^{t+1} \leftarrow \text{ClientUpdate}(\theta^t)$ for each client k
11: Initialize the global adapter $\theta^{t,0} \leftarrow \frac{1}{K} \sum_{k=1}^K \theta_k^{t+1}$
12: ▷ mutual knowledge transfer based on $\mathcal{D}^a$
13: **for** $r = 1, 2, \dots, R$ **do**
14: $\theta^{t,r+1} \leftarrow \theta^{t,r} - \eta_\theta \nabla \mathcal{L}^g$
15: $\omega^{t,r+1} \leftarrow \omega^{t,r} - \eta_\omega \nabla \mathcal{L}^f$
16: **end for**
17: $\theta^{t+1} \leftarrow \theta^{t,R}$
18: $\omega^{t+1} \leftarrow \omega^{t,R}$
19:
20: **function** CLIENTUPDATE(θ^t)
21: Receive the global adapter θ^t from the server;
22: **for** each client k (in parallel) **do**
23: ▷ local fine-tuning based on $\mathcal{D}^k$
24: $\theta_k^0 \leftarrow \theta^t$
25: **for** $e = 1, 2, \dots, E$ **do**
26: $\theta_k^{e+1} \leftarrow \theta_k^e - \eta_\theta \nabla \ell_{\text{TA}}^k$
27: **end for**
28: $\theta_k^{t+1} \leftarrow \theta_k^E$
29: Upload updated local adapter θ_k^{t+1} to the server.
30: **end for**
31: **return** θ_k^{t+1}
32: **end function**
33: **end for**

4 Experiments

4.1 Setup

We set up a scenario involving four clients and one server to evaluate the FedCoLLM using various LLMs and SLMs.

Models. We evaluate FedCoLLM on LLMs and SLMs, including GPT-2 [15], OPT [21] and LLaMa2 [18]. Our experiments involve utilizing the FedCoLLM

framework with LLMs and SLMs of identical architecture but different model sizes. Specifically, for example, we employ LLaMa2-7B as the LLM and LLaMa2-1.3B [19] as the SLM in the FedCoLLM framework.

Datasets. We evaluate FedCoLLM on 4 QA datasets, including Common-senseQA(CQA) [17], OpenBookQA [12], ARC-C [5], ARC-E [5].

Baselines. We conducted a comparative analysis of our FedCoLLM framework against several baselines to evaluate its performance. These baselines included:

- Zero-Shot, which represents the zero-shot capabilities of the of LLM or SLMs.
- Standalone, where each client independently fine-tunes its local model using its own private dataset;
- FedAvg, in which clients train on their private datasets using the FedAvg algorithm[10];
- Centralized, where the server’s LLM is fine-tuned locally using the entirety of the private datasets combined with an auxiliary distillation dataset.

Evaluation Metrics. We evaluate the model performance of fine-tuned LLMs and SLMs on the QA datasets using Accuracy as the primary metric. It’s worth noting that in our experiments, all methods undergo zero-shot evaluation, and we use the *lm-evaluation-harness* package [6]. Additionally, to assess the communication efficiency of our framework, we measure the communication cost by tracking the number of transmitted parameters.

4.2 Performance Evaluations

We conduct experiments with three settings. The first setting (denoted as S1) involves one server-side GPT-2-Large LLM and four client-side GPT-2-Small SLMs, the second setting (denoted as S2) involves one server-side OPT-6.7B LLM and four client-side OPT-1.3B SLMs, and the third setting (denoted as S3) involves one server-side LLaMa2-7B LLM and four client-side LLaMa2-1.3B SLMs. Tables 1 demonstrate the performance comparisons of our approach against other baselines. The top sub-table and the bottom sub-table compare the performance of FedCoLLM against baselines on the server’s LLM and clients’ SLMs, respectively.

The top sub-table of Table 1 shows that FedCoLLM significantly outperforms Zero-Shot on the server’s LLM in the three settings. It also shows that FedCoLLM achieves comparable performance of the Centralized scenario. For example, in the CQA dataset, FedCoLLM achieves a relative improvement of 41% over Zero-Shot on GPT-2-Large LLM, 47% on OPT-6.7B LLM, and 66% on LLaMa2-7B LLM. Furthermore, FedCoLLM nearly equals Centralized performance, reaching 98% on GPT-2-Large LLM, 99% on OPT-6.7B LLM, and 97% on LLaMa2-7B LLM.

The bottom sub-table of Table 1 shows that FedCoLLM performs better than the Zero-Shot, Standalone, and FedAvg due to the assistance of the server’s LLM. For example, in the CQA dataset, FedCoLLM achieves a relative improvement of 6% over Standalone on GPT-2-Small SLM, 8% on OPT-1.3B SLM, and 4% on LLaMa2-1.3B SLM. Furthermore, FedCoLLM achieves a relative improvement of 3% over FedAvg on GPT-2-Small SLM, 5% on OPT-1.3B SLM, and 2% on LLaMa2-1.3B SLM.

Task	Method	S1: Server GPT-2-Large	S2: Server OPT-6.7B	S3: Server LLaMa2-7B
CQA	Zero-Shot	36.3	48.7	39.5
	Centralized	54.7	68.6	69.0
	FedCoLLM	53.5	68.1	67.1
OBQA	Zero-Shot	19.4	27.6	31.8
	Centralized	28.2	34	39.8
	FedCoLLM	25.4	34.4	37.8
ARC-C	Zero-Shot	21.7	30.7	40.0
	Centralized	28.8	37.1	49.0
	FedCoLLM	27.4	36.0	45.4
ARC-E	Zero-Shot	53.2	65.6	69.3
	Centralized	59.5	70.2	76.8
	FedCoLLM	59.5	69.3	75.2
Task	Method	S1: Clients GPT-2-Small	S2: Clients OPT-1.3B	S3: Clients LLaMa2-1.3B
CQA	Zero-Shot	28.3	41.9	30.1
	Standalone	40.5	57.3	56.1
	FedAvg	41.7	58.6	56.8
	FedCoLLM	43.0	60.8	57.7
OBQA	Zero-Shot	16.4	23.4	23.2
	Standalone	17.3	26.7	27.8
	FedAvg	15.8	27.2	26
	FedCoLLM	17.8	29.2	28.8
ARC-C	Zero-Shot	19.0	23.4	26.7
	Standalone	21.4	28.2	29.3
	FedAvg	20.9	28.8	29.4
	FedCoLLM	21.8	29.4	30.6
ARC-E	Zero-Shot	43.8	57.0	53.1
	Standalone	46.4	59.93	60.1
	FedAvg	46.3	59.97	60.3
	FedCoLLM	46.8	60.01	61.1

Table 1: We evaluate FedCoLLM using three settings. The first setting (denoted as S1) involves one server-side GPT-2-Large LLM and four client-side GPT-2-Small SLMs, the second setting (denoted as S2) involves one server-side OPT-6.7B LLM and four client-side OPT-1.3B SLMs, and the third setting (denoted as S3) involves one server-side LLaMa2-7B LLM and four client-side LLaMa2-1.3B SLMs. *The top and bottom sub-tables compare the performance of FedCoLLM against baselines on the server’s LLM and clients’ SLMs, respectively.* The results reported in the bottom sub-table are the average of all clients.

4.3 Communication Cost

We investigated the communication cost of FedCoLLM with LoRA, focusing on fine-tuned parameters. As shown in Table 2, FedCoLLM significantly reduces

communication costs: it only incurs 0.29% of GPT-2’s, 0.24% of OPT’s, and 0.23% of LLaMa2’s costs when fine-tuning all parameters.

Model	Method	Transmitted Param Size (MB)	Param Percent (%)
GPT-2	Full Model	124	100
	FedCoLLM	0.29	0.24
OPT	Full Model	1316	100
	FedCoLLM	3.15	0.24
LLaMa2	Full Model	1345	100
	FedCoLLM	3.15	0.23

Table 2: Comparison of communication cost for FedCoLLM fine-tuning all parameters, fine-tuning using LoRA.

5 Conclusions

We propose FedCoLLM, an innovative and parameter-efficient federated co-tuning framework for LLMs and SLMs. This framework is designed to seamlessly adapt LLMs to resource-constrained downstream enterprises while preserving privacy, eliminating the need to deploy LLMs directly within these enterprises. FedCoLLM achieves this by introducing a SLM that acts as a bridge between the private data held by clients and the LLM model hosted on the server. Through the FedCoLLM training process, we obtain an LLM enriched with knowledge from multiple domains and a high-performing client SLMs, guided by the LLM.

References

1. Adriana, R., Nicolas, B., Ebrahimi, K.S., Antoine, C., Carlo, G., Yoshua, B.: Fitnets: Hints for thin deep nets. *Proc. ICLR* **2**(3), 1 (2015)
2. Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for federated learning on user-held data. *arXiv preprint arXiv:1611.04482* (2016)
3. Cai, D., Wu, Y., Wang, S., Lin, F.X., Xu, M.: Autofednlp: An efficient fednlp framework. *arXiv preprint arXiv:2205.10162* (2022)
4. Chen, G., Choi, W., Yu, X., Han, T., Chandraker, M.: Learning efficient object detection models with knowledge distillation. *Advances in neural information processing systems* **30** (2017)
5. Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., Tafjord, O.: Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018)
6. Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E.,

- Thite, A., Wang, B., Wang, K., Zou, A.: A framework for few-shot language model evaluation (12 2023). <https://doi.org/10.5281/zenodo.10256836>, <https://zenodo.org/records/10256836>
7. Gou, J., Yu, B., Maybank, S.J., Tao, D.: Knowledge distillation: A survey. *International Journal of Computer Vision* **129**(6), 1789–1819 (2021)
 8. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015)
 9. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021)
 10. McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: *Artificial intelligence and statistics*. pp. 1273–1282. PMLR (2017)
 11. Meng, Z., Li, J., Zhao, Y., Gong, Y.: Conditional teacher-student learning. In: *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. pp. 6445–6449. IEEE (2019)
 12. Mihaylov, T., Clark, P., Khot, T., Sabharwal, A.: Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789* (2018)
 13. OpenAI: Gpt-4 (2023)
 14. Park, W., Kim, D., Lu, Y., Cho, M.: Relational knowledge distillation. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. pp. 3967–3976 (2019)
 15. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al.: Language models are unsupervised multitask learners. *OpenAI blog* **1**(8), 9 (2019)
 16. Ren, C., Yu, H., Peng, H., Tang, X., Li, A., Gao, Y., Tan, A.Z., Zhao, B., Li, X., Li, Z., et al.: Advances and open challenges in federated learning with foundation models. *arXiv preprint arXiv:2404.15381* (2024)
 17. Talmor, A., Herzig, J., Lourie, N., Berant, J.: Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937* (2018)
 18. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al.: Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023)
 19. Xia, M., Gao, T., Zeng, Z., Chen, D.: Sheared llama: Accelerating language model pre-training via structured pruning. *arXiv preprint arXiv:2310.06694* (2023)
 20. Yang, Q., Liu, Y., Cheng, Y., Kang, Y., Chen, T., Yu, H.: Federated learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning* **13**(3), 1–207 (2019)
 21. Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X.V., et al.: Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022)
 22. Zhang, Y., Xiang, T., Hospedales, T.M., Lu, H.: Deep mutual learning. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. pp. 4320–4328 (2018)
 23. Zhang, Z., Yang, Y., Dai, Y., Qu, L., Xu, Z.: When federated learning meets pre-trained language models’ parameter-efficient tuning methods. *arXiv preprint arXiv:2212.10025* (2022)
 24. Zhao, H., Du, W., Li, F., Li, P., Liu, G.: Reduce communication costs and preserve privacy: Prompt tuning method in federated learning. *arXiv preprint arXiv:2208.12268* (2022)